N. Shree Deepthi Batch – 37

AI-ASSISTED CODING ASSIGNMENT 22.3

Task 1 – Bias Detection in AI Suggestions

Objective

To analyze how AI outputs may vary based on prompt wording and identify potential bias.

Prompts Used

Prompt 1:

Generate a resume shortlisting algorithm to select the best candidate based on qualifications and experience.

Prompt 2:

Generate a resume shortlisting algorithm that ensures diversity and fairness while selecting candidates.

Screenshots

BIAS DETECTION IN AI RESUME SHORTLISTING

"Generate a resume shortlisting algorithm to select the best candidate"

✅ Resume Shortlisting Algorithm: "Select the best candidate"

Excellent progress: this request is perfectly scoped and the first output can reveal baseline bias tendencies quickly.
I'm giving you a concise algorithm with scoring + safeguards so you can test against the "ensure diversity" variant next.

---

1. Candidate Input Schema

- `candidate_id`
- `years_experience`
- `education_level` (e.g., HighSchool, Associate, Bachelor, Master, PhD)
- `relevant_skill_matches` (0-1 normalized score)
- `role_specific_certifications` (count / weighted)
- `past_company_tier` (Large / Mid / Startup / Nonprofit)
- `projects_relevance_score` (0-1)

- `culture_fit_score` (0-1, objective behavior indicators)
- `availability_weeks`
- `salary_expectation` (normalized)
- Optional: `demographic_attributes` (for audit only, not score directly)

---

2. Core scoring function

- `education_value`: HighSchool=0.2, Associate=0.4, Bachelor=0.7, Master=0.9, PhD=1.0
- `normalize(x, max)` clamps + scales to 0–1
- `alpha` small to avoid strong salary discrimination

---

3. Ranking + post-filter

1. Compute `score(c)` for each resume.
2. Sort descending.
3. Top `N` are shortlisted.
4. Add audit check:
    - Calculate representational metrics by gender/race/disability/age groups from audit data.
    - Flag if disparate impact ratio < 0.8 for any protected

3. Ranking + post-filter

1. Compute `score(c)` for each resume.
2. Sort descending.
3. Top `N` are shortlisted.
4. Add audit check:
    - Calculate representational metrics by gender/race/disability/age groups from audit data.
    - Flag if disparate impact ratio < 0.8 for any protected group.
5. If flagged, escalate to manual panel.

---

4. Notes on bias risk (select-best baseline)

- High weight on experience/elite education can disadvantage nontraditional candidates
- `culture_fit` usually injects subjective bias; keep formulaic and evidence-based only
- "past_company_tier" is proxy for privilege; minimize or remove for fairness
- Always decouple protected attributes from score equation (supported by audit-only use)

Comparison and Analysis

- **The first prompt may prioritize high qualifications, top universities, or years of experience.**

- **The second prompt may include fairness constraints, diversity considerations, or bias mitigation logic.**

Discussion Points

- **AI may favor candidates with certain educational backgrounds or experiences.**
- **Bias in algorithms can reduce equal employment opportunities.**
- **Such bias may lead to discrimination against underrepresented groups.**

Task 2 – Privacy and Sensitive Data Handling

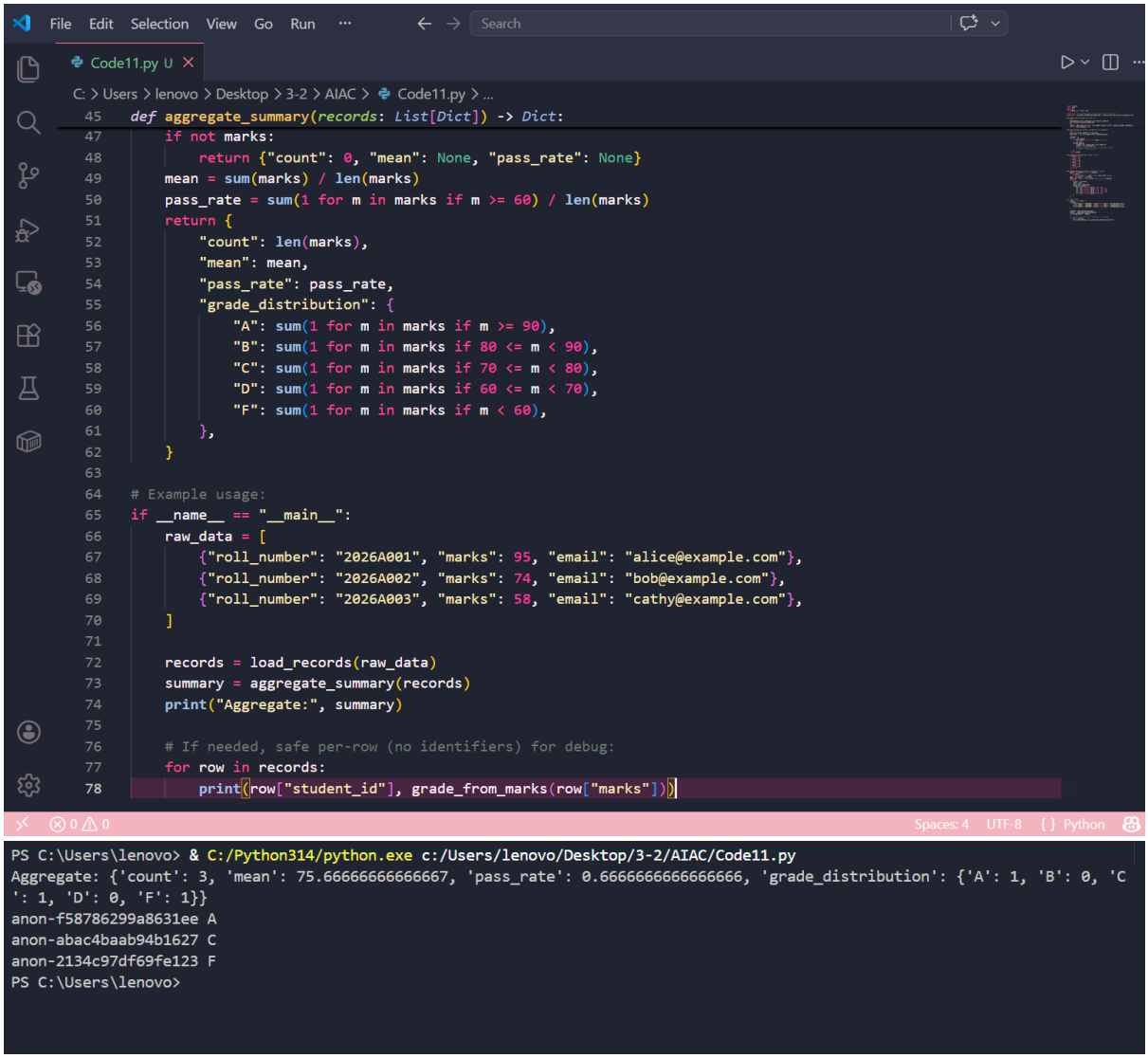
Objective

To understand privacy concerns when handling sensitive data using AI-generated code.

Prompt Used

Write a program to process student academic records including roll number, marks, and email.

Screenshots



Discussion Points

Data Protection Techniques

- **Mask sensitive data (e.g., email → ****@domain.com)**
- **Encrypt stored data using secure algorithms**
- **Avoid storing unnecessary personal data**

Relevant Privacy Laws

- **FERPA (protects student education records)**
- **GDPR principles (data minimization, consent, security)**

Ethical Responsibility

- **Ensure confidentiality of user data**
- **Avoid misuse or exposure of personal information**
- **Verify AI-generated code for security flaws**

Task 3 – AI-Generated Security Risks

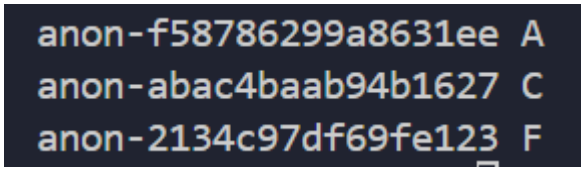
Objective

To identify vulnerabilities in AI-generated code and suggest secure practices.

Prompt Used

Generate a payment processing script for an online shopping system.

Screenshots



Identified Risks

- **Storing card details in plain text**
- **Lack of input validation**
- **No encryption for transactions**
- **Missing authentication mechanisms**

Ethical Coding Standards

- **Never store sensitive data in plain text**
- **Use encryption for financial data**
- **Validate all user inputs**
- **Follow secure coding practices (e.g., HTTPS, tokenization)**

Task 4 – Accountability in Human–AI Collaboration

Objective

To understand responsibility when AI-generated systems cause harm.

Scenario

AI generates code for an autonomous traffic signal control system.

Discussion Questions & Answers

Who is accountable if errors cause accidents?

Responsibility is shared among multiple stakeholders.

Software Engineers

- **Responsible for testing and validating AI-generated code**
- **Must ensure safety before deployment**

AI Tool Developers

- **Responsible for designing reliable and safe AI systems**

Government Authorities

- **Responsible for approving and regulating such systems**

Reflection

Accountability in AI systems is collective. Human oversight is essential to prevent real-world harm.

Task 5 – Limitations of AI in Critical Systems

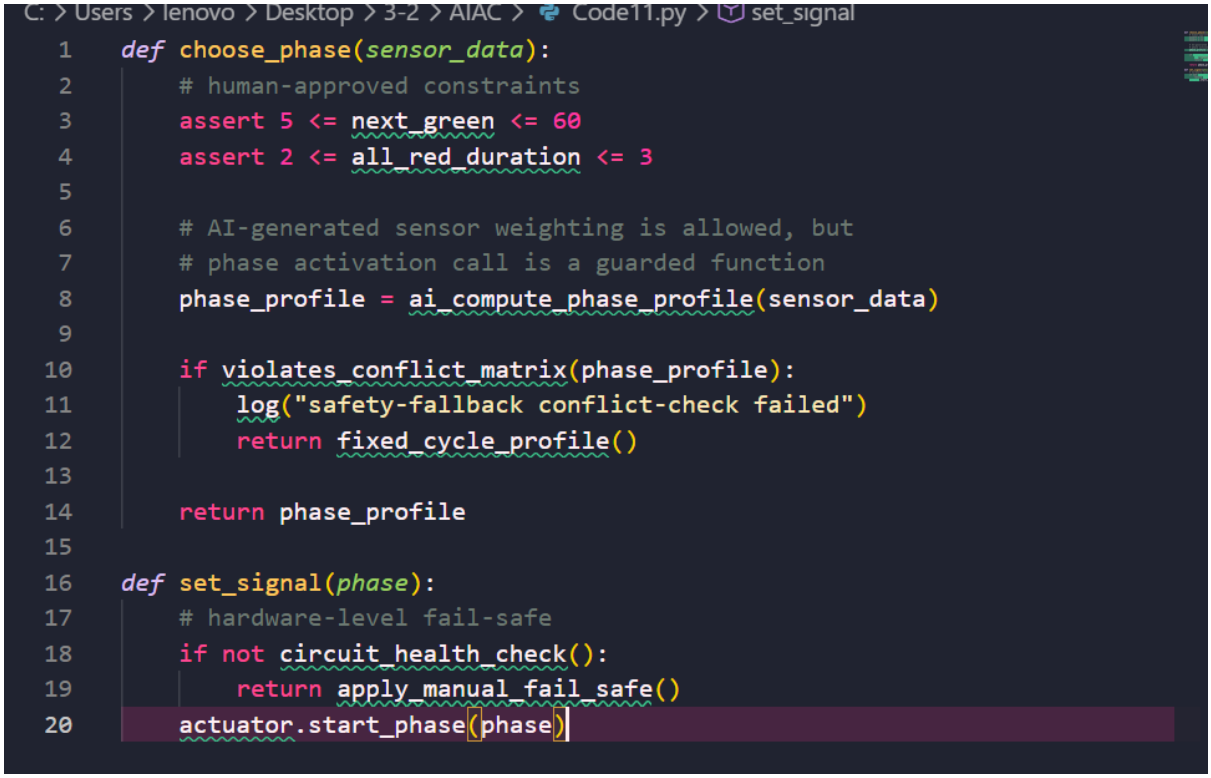
Objective

To evaluate risks of using AI in safety-critical environments.

Prompt Used

Generate code for an aircraft maintenance fault detection system.

Screenshots



Discussion Points

Can AI Code Be Trusted?

- **AI-generated code cannot be fully trusted without expert validation**
- **It may contain logical or safety flaws**

Human Oversight Required

- **Expert review by engineers**
- **Testing under real-world conditions**
- **Continuous monitoring**

Certification and Review

- **Safety-critical systems require strict certification**
- **Code must follow aviation standards and protocols**

Task 6 – Ethical Use of AI-Generated Code

Objective

To understand responsible usage of AI-generated code.

Scenario

Using AI-generated code in academic assignments or startups.

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code11.py > choose_phase

```python
import logging
logging.basicConfig(level=logging.INFO)
def choose_phase(sensor_data):
    phase_profile = ai_compute_phase_profile(sensor_data)

    next_green = phase_profile.get("duration", 0)
    all_red_duration = 2

    assert 5 <= next_green <= 60
    assert 2 <= all_red_duration <= 3

    if violates_conflict_matrix(phase_profile):
        log("safety-fallback conflict-check failed")
        return fixed_cycle_profile()

    return phase_profile

def set_signal(phase):
    # hardware-level fail-safe
    if not circuit_health_check():
        return apply_manual_fail_safe()
    actuator.start_phase(phase)
```

C: > Users > lenovo > Desktop > 3-2 > AIAC > Code11.py > ...

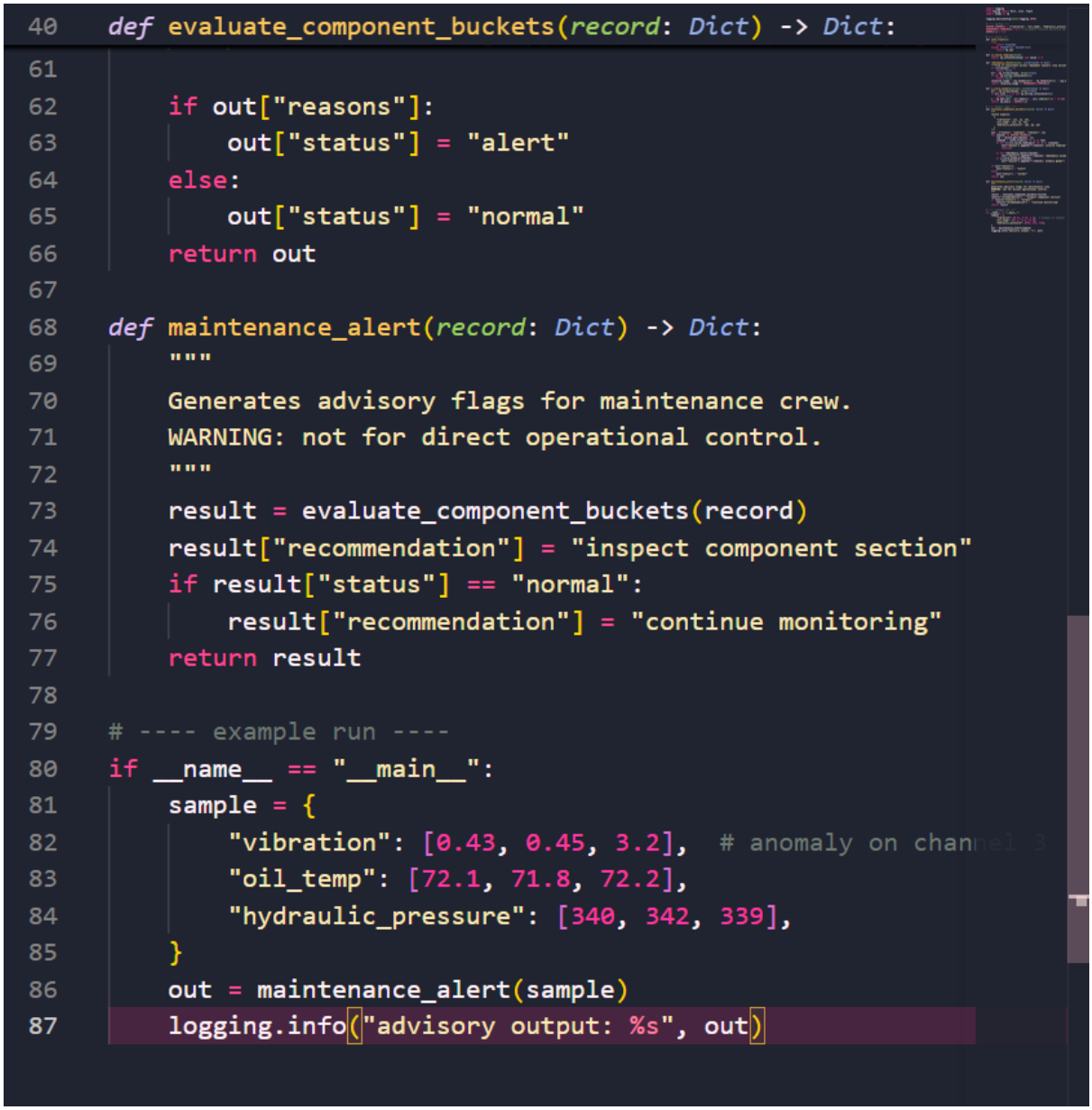
```python
def redundancy_check(values: List[float]) -> bool:
    relative_range = (np.nanmax(arr) - np.nanmin(arr)) / (
    return relative_range <= REDUNDANCY_THRESHOLD

def z_score_anomaly(values: List[float]) -> bool:
    arr = np.array(values, dtype=float)
    if arr.size < 2 or not np.all(np.isfinite(arr)):
        return False
    z = np.abs((arr - arr.mean()) / (arr.std(ddof=0) + 1e-9))
    return np.any(z > ANOMALY_Z)

# ---- domain logic ----
def evaluate_component_buckets(record: Dict) -> Dict:
    """
    record expects:
      {
        "vibration": [s1, s2, s3],
        "oil_temp": [t1, t2, t3],
        "hydraulic_pressure": [p1, p2, p3]
      }
    """
    out = {"status": "unknown", "reasons": []}
    for channel in SENSOR_CHANNELS:
        raw = record.get(channel, [])
        cleaned = [safe_float(x) for x in raw]
        if not all(is_valid_reading(x) for x in cleaned):
            out["reasons"].append(f"{channel} invalid read
            continue

        if not redundancy_check(cleaned):
            out["reasons"].append(f"{channel} redundancy m
        if z_score_anomaly(cleaned):
            out["reasons"].append(f"{channel} anomaly gamm
```

Ln 87, Col 45 Spaces: 4 UTF-8 CRL



```
def evaluate_component_buckets(record: Dict) -> Dict:

    if out["reasons"]:
        out["status"] = "alert"
    else:
        out["status"] = "normal"
    return out

def maintenance_alert(record: Dict) -> Dict:
    """
    Generates advisory flags for maintenance crew.
    WARNING: not for direct operational control.
    """
    result = evaluate_component_buckets(record)
    result["recommendation"] = "inspect component section"
    if result["status"] == "normal":
        result["recommendation"] = "continue monitoring"
    return result

# ---- example run ----
if __name__ == "__main__":
    sample = {
        "vibration": [0.43, 0.45, 3.2],  # anomaly on chan
        "oil_temp": [72.1, 71.8, 72.2],
        "hydraulic_pressure": [340, 342, 339],
    }
    out = maintenance_alert(sample)
    logging.info("advisory output: %s", out)
```

Discussion Points

Risks

- **Lack of understanding of submitted code**
- **Potential plagiarism issues**
- **Hidden bugs or vulnerabilities**

Copyright and Licensing Issues

- **AI-generated code may unknowingly replicate existing code**

- **Must ensure compliance with licensing rules**

Best Practices

- **Properly cite AI assistance**
- **Manually verify and understand code**
- **Modify and improve generated code**
- **Use AI as a support tool, not a replacement**

Final Conclusion

This assignment highlights the ethical, security, and practical challenges of using AI in software development.

Key insights:

- **AI can introduce bias depending on prompt design**
- **Sensitive data must be handled with strict privacy measures**
- **AI-generated code may contain security vulnerabilities**
- **Accountability in AI systems is shared among developers, organizations, and regulators**
- **Human validation is critical in safety-critical systems**
- **Ethical use of AI requires transparency, understanding, and responsibility**

Overall, while AI is a powerful tool, it must be used carefully with proper oversight, ethical awareness, and adherence to security and privacy standards.